Naval Research Laboratory
Washington, DC 20375-5320

NRL/6790/MR--2025/1

Collisional-Radiative Models and the Electron Boltzmann Equation

GEORGE PETROV

Directed Energy Physics Branch
Plasma Physics Division

TZVETELINA PETROVA

Radiation Hydrodynamics Branch
Plasma Physics Division

April 23, 2025

DISTRIBUTION STATEMENT A: Approved for public release; distribution is unlimited.

REPORT DOCUMENTATION PAGE

PLEASE DO NOT RETURN YOUR FORM TO THE ABOVE ORGANIZATION

1. REPORT DATE	2. REPORT TYPE	3. DATES COVERED	
23-04-2025	NRL Memorandum Report	START DATE	END DATE

4. TITLE AND SUBTITLE
Collisional-Radiative Models and the Electron Boltzmann Equation

5a. CONTRACT NUMBER	5b. GRANT NUMBER	5c. PROGRAM ELEMENT NUMBER
5d. PROJECT NUMBER	**5e. TASK NUMBER**	**5f. WORK UNIT NUMBER**
		61A1H9

6. AUTHOR(S)
George Petrov and Tzvetelina Petrova

7. PERFORMING ORGANIZATION / AFFILIATION NAME(S) AND ADDRESS(ES)
U.S. Naval Research Laboratory
4555 Overlook Avenue, SW
Washington, DC 20375

8. PERFORMING ORGANIZATION REPORT NUMBER
NRL/6790/MR--2025/1

9. SPONSORING / MONITORING AGENCY NAME(S) AND ADDRESS(ES)	10. SPONSOR / MONITOR'S ACRONYM(S) NUMBER	11. SPONSOR / MONITOR'S REPORT NUMBER(S)
	NRL	

12. DISTRIBUTION / AVAILABILITY STATEMENT
DISTRIBUTION STATEMENT A: Approved for public release; distribution is unlimited.

13. SUPPLEMENTAL NOTES

14. ABSTRACT

15. SUBJECT TERMS

16. SECURITY CLASSIFICATION OF:			17. LIMITATION OF ABSTRACT	18. NUMBER OF PAGES
a. REPORT	b. ABSTRACT	c. THIS PAGE		
U	U	U	SAR	20

19a. NAME OF RESPONSIBLE PERSON	19b. PHONE NUMBER *(Include area code)*
Dr. George Petrov	(202) 404-3970

This page intentionally left blank.

CONTENTS

1. INTRODUCTION 1
   1.1 Statement of the problem: the electron Boltzmann equation and its implementation in collisional-radiative models for low-temperature plasmas applications. 1
   1.2 Contributions of the research 2
   1.3 The approach 2
2. THE ELECTRON BOLTMANN EQUATION. STRUCTURE AND NUMERICAL SOLUTION. 2
   2.1 The time-dependent electron Boltzmann equation 2
   2.2 Input and initialization. 3
   2.3 Numerical solution. 4
   2.4 The class implementation. 5
   2.5 Numerical details. 6
   2.6 Electron particle and power balances. 7
   2.7 Steady-state versus time-dependent solutions. 8
   2.8 Comments. 8
3. SIMULATION RESULTS 8
4. APPENDIX 10
5. ACKNOWLEDGEMENTS 13
6. PUBLICATIONS IN PEER-REVIEWED JOURNALS 14
7. REFERENCES 15

FIGURES

Fig. 1 – Collisional-radiative model with a Boltzmann code

Fig. 2 – Shifted energy term interpolation

Fig. 3 – Discretization of Equation (3) is steady-state (left) and time-dependent (right). The cross denotes the discretization point.

Fig. 4 – EEDF for various RF input powers.

Fig. 5 –Plasma parameters versus input RF power.

EXECUTIVE SUMMARY

This report presents research in low-temperature and hypersonic plasmas conducted by George M. Petrov (code 6791) and Tzvetelina B. Petrova (code 6721) over the last two decades. Unique theoretical capabilities for modeling non-equilibrium plasmas have been developed, based on their research. The technique for solving collisional-radiative models coupled self-consistently to the electron Boltzmann equation is presented, together with various pieces of code that will enable the development of other codes of similar nature.

The main benefits of this research are:

- better understanding of non-equilibrium plasmas for novel Navy and DoD applications involving pulsed power, atomic layer deposition, gas lasers, plasma jets and hypersonic plasmas.
- guiding experiments and using it as a predicting tool.

The project has contributed to numerous other research programs at NRL.

This page intentionally left blank.

THE ELECTRON BOLTZMANN EQUATION IN COLLISONAL-RADIATIVE MODELS OF LOW TEMPERATURE PLASMAS

1. INTRODUCTION

1.1 Statement of the problem: the electron Boltzmann equation and its implementation in collisional-radiative models for low-temperature plasmas applications.

Low-temperature plasmas have ubiquitous applications, from gas lasers to plasma processing of materials to the semiconductor industry. The Naval Research Laboratory (NRL) has conducted extensive research to address contemporary needs that arise in low-temperature plasma applications. The research has addressed both the plasma and material side, with the plasma being in the main focus of the investigation. For modeling and simulation purposes, collisional radiative models of non-equilibrium plasmas have been developed, which generally consist of electron and heavy particle kinetics and when needed, plasma-surface interactions. The ultimate goal is to construct plasma chemistry models that have predictive capabilities. A broad range of discharges can be modeled: direct current, radio frequency, microwave, electron-beam etc.

The plasma-chemistry models are often based on fluid description of both electrons and heavy species (mass, continuity and momentum equations), which have the virtue of being computationally fast. They assume Maxwellian electron energy distribution function (EEDF) and pre-computed rates. It is well known, however, that in most gases such as Ar, nitrogen and other molecular gases, the EEDF may strongly deviate from Maxwellian. The tail of the EEDF is depleted lowering the collisional rates for electron impact excitation, dissociation and ionization. In contrast, a Maxwellian EEDF overestimates these rates. As a result, the model can give incorrect results. Correct electron collision rates can only be computed from the EEDF, which is obtained by solving the electron Boltzmann equation in some way, either directly or, *e.g.* using Monte Carlo simulations.

The concept of constructing a collisional-radiative model is schematically illustrated in Fig. 1. The electron Boltzmann equation is solved, the EEDF is calculated, then it is used to compute collisional rates with electrons. These rates, together with heavy-particle collision rates, are used to solve the species balance equations to determine the species densities. The species densities are inserted back into the electron Boltzmann equation, completing the loop. Iterations are required to achieve convergence.

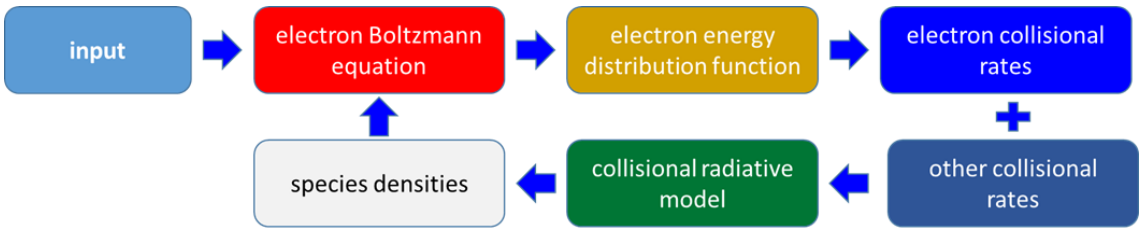


Fig. 1 – Collisional-radiative model with a Boltzmann code.

The purpose of this technical report is to provide a detailed description for solving the electron Boltzmann equation, by far the most difficult part of any collisional-radiative model. It should be noted that there are open-source Boltzmann codes, e.g. BOLSIG+ [1] or The LisbOn KInetics Boltzmann solver [2]. The main issue of open-source codes is that they are not be coupled to the chemistry model and may not

Manuscript approved April 22, 2025.

provide self-consistent description. They are generic and limited to modeling a few conventional plasma configurations. In contrast, our Boltzmann code is inherently coupled to the collisional-radiative model. More importantly, it has a modular structure that is easy to use and can implement a large number of collisional processes. For example, electron impact excitation from Ar(3p) level (the ground state) to the Ar(4s) level (excited state), can be added using a single line of code:

```
exc[++Max_exc]=excitation(Ar_3p,Ar_4s,Sigma3p_4s);
```

where the parameters in the bracket are the initial level, final level and collisional cross section, and Max_exc is a counter for the number of collisional excitation processes. Obviously, one can add as many processes as needed; there are no limitations.

1.2 Contributions of the research

The main scientific and technical contributions of this research are:

- a better understanding of low-temperature plasmas for Navy and DoD applications
- the ability to construct numerical models that support and guide experimental studies.

1.3 The approach

In general, the goal of our research is to develop capabilities to model low-temperature plasmas using a kinetics approach. In Section 2, the electron Boltzmann equation will be written in form that can be solved using conventional finite-difference approach with emphasis on its numerical structure. In particular, we will show how atomic data are inserted into the Boltzmann equation and processed using c++ classes. Some aspects of the numerical solution will be discussed.

2. THE ELECTRON BOLTMANN EQUATION. STRUCTURE AND NUMERICAL SOLUTION.

This section describes in detail the methods and procedures for solving the time-dependent electron Boltzmann equation. Subroutines and pieces of code are provided for clarity and discussion.

2.1 The time-dependent electron Boltzmann equation

The time-dependent electron Boltzmann equation is solved by two-term Legendre polynomial expansion of the Electron Velocity Distribution Function, $f(\vec{v}) = f_0(v) + \cos(\theta) f_1(v)$ [3,4], where v is the magnitude of the electron velocity and θ is he angle between the electron velocity and the electric field. The term $f_0(v)$ is the symmetrical (isotropic) part of the distribution function and $f_1(v)$ is the anisotropic part, which describes the directed motion of the electrons along the electric field. Another approximation is made by assuming that the anisotropic component of the EEDF follows adiabatically the isotropic one, *i.e.* $\frac{\partial f_1(v)}{\partial t} = 0$, which is known as the quasi-stationary approximation. This approximation is justified if the momentum transfer collision time is small compared to the typical evolution of the EEDF. With these assumptions, the electron Boltzmann equation can be written in form [3-5]:

$$\varepsilon^{1/2}\frac{\partial f_0}{\partial t} = C^{Joule}(f_0) + \sum_\alpha\left[C_\alpha^{mom}(f_0) + C_\alpha^{exc}(f_0) + C_\alpha^{ion}(f_0) + C_\alpha^{rec}(f_0)\right] + C^{Coul}(f_0) + C^{dif}(f_0), \quad (1)$$

where ε is the electron kinetic energy. The right hand side of Equation (1) contains the Joule heating term and numerous collision integrals due to both elastic and inelastic collisions with species α:

$$C^{Joule}(f_0) = \frac{2e_0}{3m_e}E_0^2\frac{\partial}{\partial\varepsilon}\left(\varepsilon^{3/2}\frac{\nu(\varepsilon)}{\nu^2(\varepsilon)+\omega^2}\frac{\partial f_0(\varepsilon,t)}{\partial\varepsilon}\right) \quad (2a)$$

$$C_\alpha^{mom}(f_0) = \frac{2m_e}{M_\alpha}\frac{\partial}{\partial\varepsilon}\left\{\varepsilon^{3/2}\nu_\alpha^{mom}(\varepsilon)\left(f_0(\varepsilon,t) + T_g\frac{\partial f_0(\varepsilon,t)}{\partial\varepsilon}\right)\right\} \quad (2b)$$

$$C^{Coul}(f_0) = \frac{\partial}{\partial\varepsilon}\left(2\Gamma^{ee}g(\varepsilon)\frac{\partial f_0(\varepsilon,t)}{\partial\varepsilon} + 3\Gamma^{ee}h(\varepsilon)f_0(\varepsilon,t)\right) \quad (2c)$$

$$C_\alpha^{exc}(f_0) = \sum_\beta \nu_{\alpha\beta}^{exc}\left(\varepsilon + \varepsilon_{\alpha\beta}^{exc}\right)\left(\varepsilon + \varepsilon_{\alpha\beta}^{exc}\right)^{1/2} f_0\left(\varepsilon + \varepsilon_{\alpha\beta}^{exc}, t\right) - \nu_{\alpha\beta}^{exc}(\varepsilon)\varepsilon^{1/2} f_0(\varepsilon, t) \quad (2d)$$

$$C_\alpha^{ion}(f_0) = \sum_\beta 4\nu_{\alpha\beta}^{ion}\left(2\varepsilon + \varepsilon_{\alpha\beta}^{ion}\right)\left(2\varepsilon + \varepsilon_{\alpha\beta}^{ion}\right)^{1/2} f_0\left(2\varepsilon + \varepsilon_{\alpha\beta}^{ion}, t\right) - \nu_{\alpha\beta}^{ion}(\varepsilon)\varepsilon^{1/2} f_0(\varepsilon, t) \quad (2e)$$

$$C_\alpha^{rec}(f_0) = -\nu_\alpha^{rec}(\varepsilon)\varepsilon^{1/2} f_0(\varepsilon, t) \quad (2f)$$

$$C^{dif}(f_0) = -\nu^{dif}\varepsilon^{1/2} f_0(\varepsilon, t). \quad (2g)$$

In Equations (1) and (2), the energy ε, energy threshold $\varepsilon_{\alpha\beta}^{\mu}$ and the gas temperature T_g are in units of electronvolts (eV). The energy-dependent rates for inelastic collisions are $\nu_{\alpha\beta}^{\mu}(\varepsilon) = \sigma_{\alpha\beta}^{\mu}(\varepsilon)\nu_e(\varepsilon)n_\alpha$, where superscript μ denotes the collision process, $\sigma_{\alpha\beta}^{\mu}(\varepsilon)$ is the collision cross section for transition from species α to species β, which is an electronically excited state, dissociative or ionization level. The total collision frequency in Equation (2a) is $\nu(\varepsilon) = \sum_\alpha\left(\nu_\alpha^{mom}(\varepsilon) + \sum_\beta \nu_{\alpha\beta}^{\mu}(\varepsilon)\right)$. Species α has mass M_α and density n_α, $\nu_e(\varepsilon) = \sqrt{\frac{2e_0\varepsilon}{m_e}}$ is the electron velocity, and e_0 and m_e are the magnitude of the electron charge and mass, respectively. The electron-electron collisions, Equation (2c), are included using the Fokker–Planck integrals $g(\varepsilon) = \int_0^\varepsilon \varepsilon'^{3/2} f_0(\varepsilon', t)d\varepsilon' + \varepsilon^{3/2}\int_\varepsilon^\infty f_0(\varepsilon', t)d\varepsilon'$ and $h(\varepsilon) = \int_0^\varepsilon \varepsilon'^{1/2} f_0(\varepsilon', t)d\varepsilon'$ with a coefficient $\Gamma^{ee} = \sqrt{\frac{2e_0}{m_e}}\frac{e_0^2}{24\pi\varepsilon_0^2} ln\,(\Lambda)$, where $ln(\Lambda) = 12\pi\frac{(\varepsilon_0 e_0 T_e)^{3/2}}{e_0^3 n_e^{1/2}}$ is the Coulomb logarithm, n_e and T_e are the electron density and temperature (2/3 of the mean electron energy) and ε_0 is the permittivity of free space [6]. The ionization collision integral is written in form assuming that the incoming and newly born electrons share equally the remaining energy, which is justified at low collision energies. Dissociation of molecules such as N_2 are treated as excitation. The Joule heating term contains the angular frequency ω if the electric field is an RF field (for direct current discharges ω=0) and E_0 is the electric field envelope. The normalization condition for the EEDF is $\int_0^\infty \varepsilon^{1/2} f_0(\varepsilon, t)d\varepsilon = n_e(t)$.

2.2 Input and initialization.

The collisional-radiative model and the electron Boltzmann equation in particular need both input and initialization. The input is typically the gas pressure, plasma dimensions and current (for a direct current discharge), power and RF frequency (for an RF or microwave discharge), or e-beam current (for an e-beam driven discharge). The initialization includes initial species densities and EEDF, as well as prep work for solving the electron Boltzmann equation such as setting up an energy grid. Finally, the collision processes are loaded. A sample code is shown below for Ar.

```
//-------------------------------- elastic scattering -------------------------------------
// 1. Ar + e -> Ar + e
      mom[++Max_mom]=elastic(Ar,Sigma_mom,Ar_mass);
// 2. Ar+ + e -> Ar+ + e
      mom[++Max_mom]=elastic(Ar_II,Sigma_ei,Ar_mass);
// 3. Ar2+ + e -> Ar2+ + e
      mom[++Max mom]=elastic(Ar2 II,Sigma ei,2.0*Ar mass);
//---------------------------------- excitation -----------------------------------------
// 1. Ar + e -> Ar(4s) + e
      exc[++Max_exc]=excitation(Ar,Ar_4s,Sigma3p_4s);
// 2. Ar + e -> Ar(4p) + e
      exc[++Max_exc]=excitation(Ar,Ar_4p,Sigma3p_4p);
// 3. Ar(4s) + e -> Ar(4p) + e
      exc[++Max_exc]=excitation(Ar_4s,Ar_4p,Sigma4s_4p);
//---------------------------------- ionization -----------------------------------------
// 1. Ar + e -> Ar+ + 2*e
      ion[++Max_ion]=ionization(Ar,Ar_II,Sigma3p_ion);
// 2. Ar(4s) + e -> Ar+ + 2*e
      ion[++Max_ion]=ionization(Ar_4s,Ar_II,Sigma4s_ion);
// 3. Ar(4p) + e -> Ar+ + 2*e
      ion[++Max_ion]=ionization(Ar_4p,Ar_II,Sigma4p_ion);
```

```
  //----------------------------- dissociative recombination ---------------------------------
  // 1. Ar2^+ + e -> Ar(4s) + Ar
       dr[++Max_dr]=diss_recombination(Ar2_II,Ar_4s,Ar,Sigma_rec);
  //------------------------------ tri-body recombination -----------------------------------
  // 1. Ar+ + e + Ar -> Ar(4s) + Ar
       rec[++Max_rec]=recombination(Ar_II,Ar_4s,Ar,2.5e-29,-1.5);
  // 2. Ar+ + e + e -> Ar(4s) + e
       rec[++Max_rec]=recombination(Ar_II,Ar_4s,e,7.2e-27,-4.5);
  //---------------------------------- diffusion ------------------------------------------
  // 1. diffusion
       dif[++Max_dif]=diffusion(R0,beta_radial);
}
```

2.3 Numerical solution.

The electron Boltzmann equation is cast in form, suitable for numerical solution:

$$\varepsilon^{1/2}\frac{\partial f_0(\varepsilon,t)}{\partial t} = \left(a(\varepsilon)\frac{\partial f_0(\varepsilon,t)}{\partial \varepsilon} + b(\varepsilon)f_0(\varepsilon,t)\right) + c(\varepsilon)f_0(\varepsilon,t) = d(\varepsilon). \tag{3}$$

The coefficients *a* through *d* are accumulated in the classes such as the one shown in Section 2.4. The electron Boltzmann equation is a second-order time-dependent partial differential equation with nonlinear integral coefficients arising from the Fokker–Planck terms. It is discretized using the Crank-Nicholson method and solved by a second-order correct finite difference scheme using the predictor-corrector method. Boundary conditions are also required. At the last grid point (maximum energy) $f_0(\varepsilon,t) = 0$. The other boundary condition is not trivial. The normalization condition, $\int_0^\infty \varepsilon^{1/2} f_0(\varepsilon,t)d\varepsilon = n_e(t)$, is discretized at the first grid point, $\varepsilon = 0$, forming a full line [7]. Thus, the discretized version of the Boltzmann equation becomes a tri-diagonal matrix plus a full first line for *i*=0:

$$\sum_{j=0}^{n} p_j f_j = \bar{d}_0, j = 0 \ldots n \tag{4a}$$

$$\bar{a}_i f_{i-1} + \bar{b}_i f_i + \bar{c}_i f_{i+1} = \bar{d}_i, i = 1 \ldots n \tag{4b}$$

$$f_n = 0 \tag{4a}$$

The coefficients in Equation (4) are:

$$\bar{a}_i = a_{i-1/2}^{mid} - b_{i-1/2}^{mid} \tag{5a}$$

$$\bar{b}_i = -a_{i-1/2}^{mid} - b_{i-1/2}^{mid} - a_{i+1/2}^{mid} + b_{i+1/2}^{mid} + c_i^{mid} + \tau_i \tag{5b}$$

$$\bar{c}_i = a_{i+1/2}^{mid} + b_{i+1/2}^{mid} \tag{5c}$$

$$\bar{d}_i = d_i^{mid} - \bar{a}_i f_{i-1}^{(t)} - \left(\bar{b}_i - 2\tau_i\right) f_i^{(t)} - \bar{c}_i f_{i+1}^{(t)} \tag{5d}$$

where in Equation (5) the quantities are $a_{i-1/2}^{mid} = \frac{a_{i-1}^{(t)} + a_i^{(t)} + a_{i-1}^{(t+\Delta t)} + a_i^{(t+\Delta t)}}{8\Delta\varepsilon^2}$, $b_{i-1/2}^{mid} = \frac{b_{i-1}^{(t)} + b_i^{(t)} + b_{i-1}^{(t+\Delta t)} + b_i^{(t+\Delta t)}}{16\Delta\varepsilon}$, $c_i^{mid} = \frac{c_i^{(t)} + c_i^{(t+\Delta t)}}{4}$, $d_i^{mid} = \frac{d_i^{(t)} + d_i^{(t+\Delta t)}}{2}$ and $\tau_i = -\frac{\sqrt{\varepsilon_i}}{v_1 \Delta t}$, in which v_1 is the electron velocity at kinetic energy 1 eV. Superscript *mid* indicates time $t+\Delta t/2$. In spite of the full first line, the numerical method is as efficient as a conventional tri-diagonal matrix solution, see the subroutine below.

```
// solves tri-diagonal linear system of equations with full first line
void slse(double *a,double *b,double *c,double *d,double *p) {
  for (i=n-2;i>=1;i--) {
    b[i]=b[i]-a[i+1]*c[i]/b[i+1];
    d[i]=d[i]-d[i+1]*c[i]/b[i+1];
    p[i]=p[i]-a[i+1]*p[i+1]/b[i+1];
    d[0]=d[0]-d[i+1]*p[i+1]/b[i+1];
  }
  p[0]=p[0]-a[1]*p[1]/b[1];d[0]=d[0]-d[1]*p[1]/b[1];
  f[0]=d[0]/p[0];for (i=1;i<=n-1;i++) f[i]=(d[i]-a[i]*f[i-1])/b[i];
}
```

2.4 The class implementation.

The non-trivial part is the assembling of the coefficients *a* to *d*, which will be explained next. It is not uncommon to include a large number of processes (20-100) into the electron Boltzmann equation, particularly for molecules or a mixture of gases. But, this is not all: after the EEDF f_0 is computed, the rate coefficients for all processes must be calculated and the rates must be added to the species balance equations. Clearly, this can be a colossal task, prone to errors. A simple and very safe method is to use c++ classes, one for each type of process (Joule heating, excitation, etc.). Class *excitation* is analyzed below. Other classes are listed in the Appendix. In the code, *u* denotes the kinetic energy ε.

```
class excitation {                                      // ---------- begin class ------------
  private:                                              // private variables
    int i;                                              // local counter
    double *tmp;                                        // temporary array
  public:                                               // public variables
    species InitialState;                               // initial state
    species FinalState;                                 // final state
    double *sigma_exc;                                  // excitation cross section in [cm^2]
    double *sigma_dex;                                  // deexcitation cross section in [cm^2]
    double dE;                                          // energy threshold in [eV]
    double k_exc;                                       // excitaion rate coefficient [cm^3/s]
    double k_dex;                                       // deexcitaion rate coefficient [cm^3/s]
    double R_exc;                                       // excitaion rate [cm^(-3)/s]
    double R_dex;                                       // deexcitaion rate [cm^(-3)/s]
    double P_exc;                                       // power loss [eV*cm^(-3)/s]
    double P_dex;                                       // power gain [eV*cm^(-3)/s]
    double P;                                           // net power loss [eV*cm^(-3)/s]
    excitation() {};                                    // constructor
    excitation(species InitialState,species FinalState,double S_exc(double u));
    ~excitation() {};                                   // destructor
    void AddTo_BE();                                    // add coefficients to Boltzmann eq.
    void get_rates();                                   // calculate rates
};                                                      // ----------- end class -------------
excitation::excitation(species InitialState,species FinalState,double S_exc(double u)) {
  tmp = new double[n+1];                                // reserve memory for array "tmp"
  sigma_exc = new double[n+1];                          // reserve memory for array "sigma_exc"
  sigma_dex = new double[n+1];                          // reserve memory for array "sigma_dex"
  this->InitialState=InitialState;                      // set local "InitialState"
  this->FinalState  =FinalState;                        // set local "FinalState"
  dE=E[FinalState]-E[InitialState];                     // set energy threshold in [eV]
  for (i=0;i<=n;i++) {                                  // begin loop over "i"
    sigma_exc[i]=S_exc(u[i]);                           // map cross section on grid points
    sigma_dex[i]=S_exc(u[i]+dE)*(u[i]+dE)/(u[i]+1.0e-3)*g[InitialState]/g[FinalState];
  }                                                     // end loop over "i"
}                                                       // end constructor
void excitation::AddTo_BE() {                           // begin "AddTo_BE"
  double tmp_exc,tmp_dex;                               // local variables
  for (i=0;i<=n;i++) {                                  // begin loop over "i"
    c[i]-=sigma_exc[i]*u[i]*N[InitialState]+sigma_dex[i]*u[i]*N[FinalState];
    tmp_exc=u[i]+dE;tmp_dex=u[i]-dE;
    d[i]-=y_shift(sigma_exc,tmp_exc)*tmp_exc*y_shift(f,tmp_exc)*N[InitialState]+
          y_shift(sigma_dex,tmp_dex)*tmp_dex*y_shift(f,tmp_dex)*N[FinalState];
    QN[i]+=sigma_exc[i]*N[InitialState]+sigma_dex[i]*N[FinalState];
  }                                                     // end loop over "i"
}                                                       // end "AddTo_BE"
void excitation::get_rates() {                          // begin "get_rates"
  for (i=0;i<=n;i++) tmp[i]=sigma_exc[i]*u[i]*f[i];     // set array "tmp"
  k_exc=Simpson(tmp)*v1;R_exc=k_exc*N[InitialState]*Ne;// rate in [cm^-3/s]
  P_exc=R_exc*dE;                                       // power loss in [eV*cm^(-3)/s]
  for (i=0;i<=n;i++) tmp[i]=sigma_dex[i]*u[i]*f[i];     // set array "tmp"
  k_dex=Simpson(tmp)*v1;R_dex=k_dex*N[FinalState]*Ne; // rate in [cm^-3/s]
  P_dex=R_dex*dE;                                       // power gain in [eV*cm^(-3)/s]
  P=P exc-P dex;                                        // net power loss in [eV*cm^(-3)/s]
  loss[InitialState]+=R_exc;gain[FinalState]+=R_exc;  // loss InitialState, gain FinalState
  loss[FinalState]+=R_dex;gain[InitialState]+=R_dex;  // loss FinalState, gain InitialState
}                                                       // end "get_rates"
excitation *exc = new excitation[MaxExc+1];             // define class variable
```

Here is what a class does:

- takes input data such as cross sections, species densities, initial and final species
- adds the corresponding term to the electron Boltzmann equation
- computes rate coefficients and rates
- adds the computed rates to the species balance equations

The constructor excitation::excitation(species InitialState, species FinalState, double S_exc(double u)) takes input data: initial and final species and collision cross section as a function of the kinetic energy. The species densities *N* are global variables visible inside the class. Subroutine excitation::AddTo_BE() adds the contribution of this process to coefficients *a*, *b*, *c* or *d*, which are arrays, as well as array QN, which is $\sigma_{tot}(\varepsilon)N$. In this case, contributions are added to coefficients *c* and *d*. Subroutine excitation::get_rates() computes the excitation and de-excitation rate coefficients k_exc and k_dex, then computes the rates R_exc and R_dex, and power gain and loss P_exc and P_dex. At the very end of the subroutine, the rates are added to the species balance equation as gain or loss. Thus, the class does all the work. Function y_shift computes $f_0\left(\varepsilon + \varepsilon_{ak}^{ion}, t\right)$ and function Simpson does numerical integration, as the name suggests.

2.5 Numerical details.

The right hand side of Equation (2) contains terms that are shifted in energy, e.g. $f_0(\varepsilon + \varepsilon_{ak}^{exc}, t)$ or $f_0\left(2\varepsilon + \varepsilon_{ak}^{ion}, t\right)$. The shifted energy, denoted as u_shifted, may fall between grid points and an interpolation using 3 grid points is performed (Figure 2):

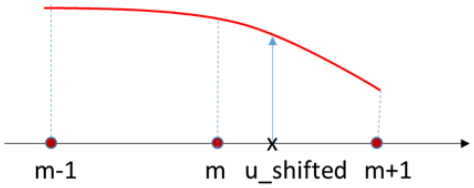


Fig. 2 – Shifted energy term interpolation.

$$f_0\left(\varepsilon_{shift}\right) = \frac{1}{2}\left(\frac{1}{2} - \Delta m\right)^2 f_{m-1} + \left(\frac{3}{4} - \Delta m^2\right) f_m + \frac{1}{2}\left(\frac{1}{2} + \Delta m\right)^2 f_{m+1} \tag{5}$$

where $\Delta m = \frac{\varepsilon_{shift}}{\Delta\varepsilon} - m$. A sample code is shown below.

```
double y_shift(double y[],double u_shifted) {
  int m;double dm;
  m=int(u_shifted/du+0.5);dm=u_shifted/du-m;
  if ((u_shifted < Umax) && (u_shifted > 0.0)) {
    switch (m) {
      case 0: return (1.0-dm)*y[0]+dm*y[1];break;
      case n: return (1.0+dm)*y[n]-dm*y[n-1];break;
      default: return 0.5*sqr(0.5-dm)*y[m-1]+(0.75-sqr(dm))*y[m]+0.5*sqr(0.5+dm)*y[m+1];
    }
  }
  else return 0.0;
}
```

The coefficients p_i in the first line are computed as follows:

$$p_0 = [1 + 2(p-1)(p-2)]q/9 \tag{6a}$$

$$p_1 = -4p(p-2)q/9 \tag{6b}$$

$$p_2 = [2.5 + 2p(p-1]q/9 \tag{6c}$$

$$p_i = \frac{4}{3}\sqrt{\varepsilon_i}\Delta\varepsilon, i = 3{,}5{,}7\ldots \tag{6d}$$

$$p_i = \frac{2}{3}\sqrt{\varepsilon_i}\Delta\varepsilon, i = 4{,}6{,}8\ldots \tag{6e}$$

where $p = 2^{1/3}$ and $q = (2\Delta\varepsilon)^{3/2}$. A sample code is shown below.

```
// calculate the coefficients for the modified Simpson's rule for the tri-diagonal linear
// system of equations with full first line: Integral{f(u)*u^(1/2)*du}=sum{p0[i]*f[i]}
void set_p() {                                                    // begin subroutine
  int i;double p,q;                                               // local variables
  p=pow(2.0,1.0/3.0);q=pow(2.0*du,1.5);                           // set "p" and "q"
  p0[0]=(1.0+2.0*(p-1)*(p-2))*q/9.0;                              // coefficient p0[0]
  p0[1]=-4.0*p*(p-2.0)*q/9.0;                                     // coefficient p0[1]
  p0[2]=(2.5+2.0*p*(p-1.0))*q/9.0;                                // coefficient p0[2]
  for (i=3;i<=n-1;i+=2) {                                         // begin loop over "i"
    p0[i]=4.0*sqrt(u[i])*du/3.0;                                  // odd "i"
    p0[i+1]=2.0*sqrt(u[i+1])*du/3.0;                              // even "i"
  }                                                               // end loop over "i"
  p0[n]=sqrt(u[n])*du;                                            // i=n
}                                                                 // end subroutine
```

Finally, the actual solution of the electron Boltzmann equation are two small subroutines, that (i) loop over all collisional processes; (ii) make a discretization by forming arrays a[i], b[i], c[i] and d[i] in Equation (4b); and (iii) solve the linear system of equations. A sample code is shown below.

```
void calc_coefficients() {
  int process;
  // form the coefficients a, b, c and d by adding processes
  for (i=0;i<=n;i++) {a[i]=0.0;b[i]=0.0;c[i]=0.0;d[i]=0.0;QN[i]=0.0;T[i]=-sqrt_u[i]/v1;}
  for (process=1;process<=Max_mom;process++) mom[process].AddTo_BE();
  for (process=1;process<=Max_exc;process++) exc[process].AddTo_BE();
  for (process=1;process<=Max_ion;process++) ion[process].AddTo_BE();
  for (process=1;process<=Max_dif;process++) dif[process].AddTo_BE();
  for (process=1;process<=Max_rec;process++) rec[process].AddTo_BE();
  for (process=1;process<=Max_dr; process++) dr[process].AddTo_BE();
  el_field->AddTo_BE();
  ee_coll->AddTo_BE();
}
void calc_eedf() {
  double tmp1,tmp2,tmp3,tmp4,tmp5,tmp6,tmp7;                  // local variables
  // make the discretization: a1[i]*f[i-1]+b1[i]*f[i]+c1[i]*f[i+1]=d1[i]
  double Eightdu2=8*du*du;double Sixteendu=16*du;             // calculate 8*du^2 and 16*du
  for (i=1;i<=n-1;i++) {                                      // begin loop over grid points
    tmp1=(a[i-1]+a[i]+a_old[i-1]+a_old[i])/Eightdu2;          // calculate a_mid(i-1/2)/(2*du^2)
    tmp2=(b[i-1]+b[i]+b_old[i-1]+b_old[i])/Sixteendu;         // calculate b_mid(i-1/2)/(4*du)
    tmp3=(a[i+1]+a[i]+a_old[i+1]+a_old[i])/Eightdu2;          // calculate a_mid(i+1/2)/(2*du^2)
    tmp4=(b[i+1]+b[i]+b_old[i+1]+b_old[i])/Sixteendu;         // calculate b_mid(i+1/2)/(4*du)
    tmp5=(c[i]+c_old[i])/4.0;                                 // calculate c_mid(i)/2
    tmp6=(d[i]+d_old[i])/2.0;                                 // calculate d_mid(i)
    tmp7=T[i]/dt;                                             // time
    a1[i]=tmp1-tmp2;                                          // calculate a1[i]
    b1[i]=tmp7+tmp5-tmp1-tmp2-tmp3+tmp4;                      // calculate b1[i]
    c1[i]=tmp3+tmp4;                                          // calculate c1[i]
    d1[i]=tmp6-a1[i]*f_old[i-1]-(b1[i]-2.0*tmp7)*f_old[i]-c1[i]*f_old[i+1];
  }                                                           // end loop over grid points
  // set the first row of the linear system of equations
  for (i=0;i<=n;i++) p1[i]=p0[i];                             // set the first line coefficients
  // set the boundary conditons:
  // 1.sum{p1[i]*f[i]}=Ne (equivalent to integral{f(u)*u^0.5*du}=Ne)
  // 2.f[n]=0
  d1[0]=Ne;f[n]=0.0;                                          // set boundary conditions
  // solve the linear system of equations to get f[i] for i=0...n
  slse(a1,b1,c1,d1,p1);                                       // calculate f(u)
  for (i=0;i<=n;i++) f[i]=max(f[i],1.0e-90);                  // make f(u) > 0
}                                                             // end subroutine
```

2.6 Electron particle and power balances.

After the Boltzmann equation is advanced one time step, the cross sections are used to calculate the corresponding rate coefficients, Joule heating term and power loss terms for individual processes due to elastic and inelastic collisions. Integration of the electron Boltzmann equation over energy yields the electron particle balance equation [4,5]:

$$\frac{\partial n_e}{\partial t} = \sum_{\alpha,\beta} R_{\alpha\beta}^{ion} - \sum_{\alpha} R_{\alpha}^{rec} - R^{dif} \tag{7}$$

Multiplication by ε and integration over energy yields the electron power balance:

$$\frac{d\langle\varepsilon(t)\rangle}{dt} = P^{Joule} - \sum_{\alpha}\left[P_{\alpha}^{mom}(t) + P_{\alpha}^{exc}(t) + P_{\alpha}^{ion}(f_0) + P_{\alpha}^{rec}(f_0)\right] - P^{dif}, \tag{8}$$

where $\langle\varepsilon\rangle = \int_0^\infty \varepsilon^{3/2} f_0(\varepsilon) d\varepsilon$ is the mean energy density of electrons and *P* denotes power gain or loss per unit volume [4,5].

2.7 Steady-state versus time-dependent solutions.

In steady-state, the discretization of Equation (3) is straightforward using three grid points: $\{u_{i-1}, u_i, u_{i+1}\}$. The time-dependent case is discretized at $\{t + \frac{1}{2}\Delta t, u_i\}$ using the Crank-Nicholson method with six grid points (Figure 3). In both cases the discretization is second-order correct and the resulting linear system of equations is of the same type: tri-diagonal plus full first line.

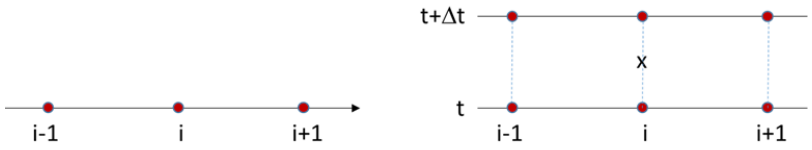


Fig. 3 – Discretization of Equation (3) is steady-state (left) and time-dependent (right). The cross denotes the discretization point.

In steady-state, the normalization of the EEDF is $\int_0^\infty \varepsilon^{1/2} f_0(\varepsilon) d\varepsilon = 1$ and the Coulomb collision terms are multiplied by n_e, while for time-dependent problems the normalization is $\int_0^\infty \varepsilon^{1/2} f_0(\varepsilon, t) d\varepsilon = n_e(t)$.

2.8 Comments.

The technique for solving the electron Boltzmann equation together with the main pieces of code are presented. One has to keep in mind that they have to be adapted to the particular problem. New classes may have to be added to account, *e.g.* electron beams [8] or dissociation. Other classes may have to be modified. In Ref. 8, both axial and radial diffusion of charged species is accounted for with diffusion along and across magnetic field.

3. SIMULATION RESULTS

Simulation results are presented for an Ar RF discharge. The runs are steady-state for simplicity. The EEDF is plotted in Figure 4. It changes considerably with power, but is far from a Maxwellian.

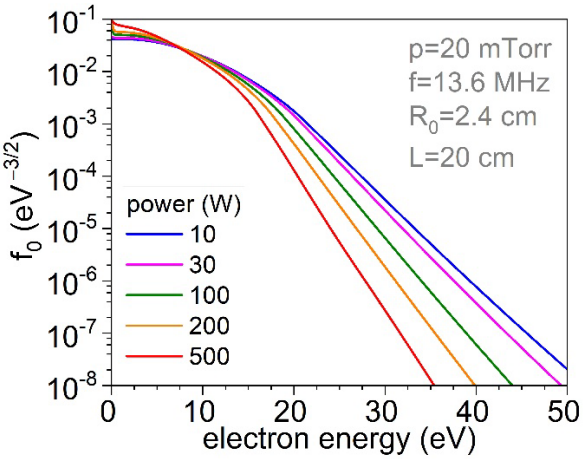


Fig. 4 – EEDF for various RF input powers.

The plasma parameters are plotted in Figure 5. The top left figure plots the electron temperature and the amplitude of the RF field E_0. The latter is computed by balancing the ionization with diffusion and recombination. E_0 is varied until $\sum_{\alpha,\beta} R_{\alpha k}^{ion} = \sum_{\alpha} R_{\alpha}^{rec} + R^{dif}$. The species densities are plotted in the right panels. The power deposition (Joule heating) and power dissipation in various channels is plotted in the bottom left panel. The dominant power loss mechanism is electron impact excitation. Diffusion power loss is also important because of the low gas pressure.

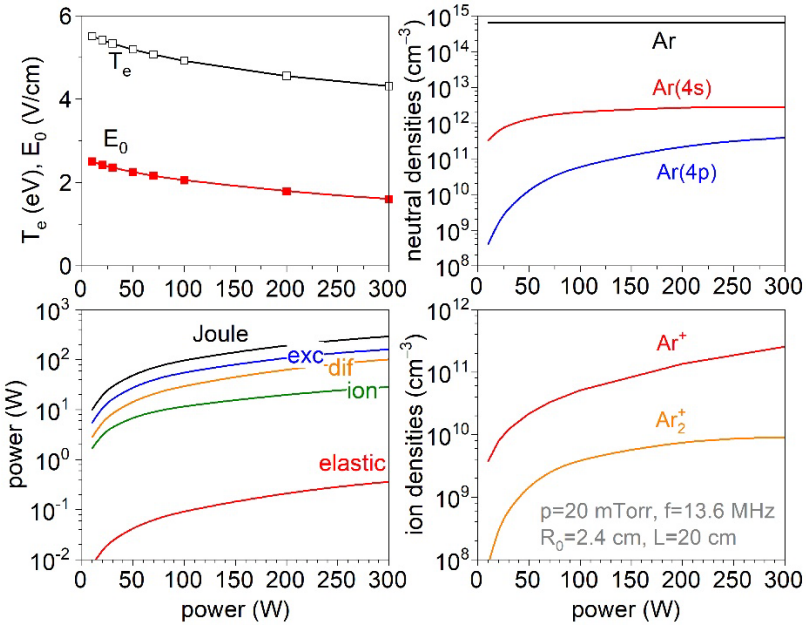


Fig. 5 – Plasma parameters versus input RF power.

4. APPENDIX

The classes are at the hearth of the Boltzmann code. Numerous classes are listed below that can help with the construction of a Boltzmann code.

```
class elastic {                                                    // begin class definition
  private:                                                         // private variables
    int i;                                                         // counter
    double *tmp;                                                   // temporary array
  public:                                                          // public variables
    species InitialState;                                          // initial state
    double two_mass_ratio;                                         // 2*me/M
    double *sigma;                                                 // momentum transfer cross section
    double k;                                                      // rate coefficient [cm^3/s]
    double R;                                                      // rate [cm^(-3)/s]
    double P;                                                      // power loss [eV*cm^(-3)/s]
    double I;                                                      // momentum in [cm/s^2]
    elastic() {};                                                  // default constructor
    elastic(species InitialState,double S_mom(double u),double mass);// constructor
    ~elastic() {};                                                 // destructor
    void AddTo_BE();                                               // add data to BE
    void get_rates();                                              // calculate rates
};                                                                 // end class definition
elastic::elastic(species InitialState,double S_mom(double u),double mass) {
  tmp = new double[n+1];                                           // reserve memory for array "tmp"
  sigma = new double[n+1];                                         // reserve memory for array "sigma"
  this->InitialState=InitialState;                                 // assign initial state
  two_mass_ratio=2.0/(1836.0*mass);                                // set 2*me/M
  for (i=0;i<=n;i++) sigma[i]=S_mom(u[i]);                         // map cross section on the grid
}                                                                  // end constructor
void elastic::AddTo BE() {
  for (i=0;i<=n;i++) {                                             // begin loop over grid points
    b[i]+=two_mass_ratio*sigma[i]*sqr(u[i])*N[InitialState];// add contribution to BE coeff.
    QN[i]+=sigma[i]*N[InitialState];                               // add contribution to sigma(u)*N
  }                                                                // end loop over grid points
}                                                                  // end AddTo_BE
void elastic::get_rates() {                                        // begin get_rates
  for (i=0;i<=n;i++) tmp[i]=sigma[i]*u[i]*f[i];                    // set array tmp
  k=Simpson(tmp)*v1/Ne;                                            // rate coefficient in [cm^3/s]
  R=k*N[InitialState]*Ne;                                          // rate in [cm^-3/s]
  for (i=0;i<=n;i++) tmp[i]=sigma[i]*sqr(u[i])*f[i];               // set array tmp
  P=two_mass_ratio*Simpson(tmp)*v1*N[InitialState];                // power loss in [eV*cm^-3/s]
  for (i=0;i<=n;i++) tmp[i]=u[i]*sqrt_u[i]*sigma[i]*f1[i];  // set array tmp[i]
  I=-sqr(v1)*Simpson(tmp)*N[InitialState]/3.0;                     // momentum loss in [cm/s^2]
}                                                                  // end get_rates
elastic *mom = new elastic[MaxMom+1];                              // class variables
```

```
class ionization {                                                 // begin class definition
  private:                                                         // private variables
    int i;                                                         // counter
    double *tmp;                                                   // temporary array
  public:                                                          // public variables
    species InitialState;                                          // initial state
    species FinalState;                                            // final state
    double *sigma;                                                 // electron impact cross section
    double threshold;                                              // energy threshold in [eV]
    double k;                                                      // rate coefficient [cm^3/s]
    double R;                                                      // rate [cm^(-3)/s]
    double P;                                                      // power loss [eV*cm^(-3)/s]
    double I;                                                      // momentum in [cm/s^2]
    ionization() {};                                               // default constructor
    ~ionization() {};                                              // destructor
    ionization(species InitialState,species FinalState,double S_ion(double u));
    void AddTo_BE();                                               // add data to BE
    void get_rates();                                              // calculate rates
};                                                                 // end class definition
ionization::ionization(species InitialState,species FinalState,double S_ion(double u)) {
  tmp = new double[n+1];                                           // reserve memory for array "tmp"
```

```
  sigma = new double[n+1];                                    // reserve memory for array "sigma"
  this->InitialState=InitialState;                            // assign initial state
  this->FinalState  =FinalState;                              // assign final state
  threshold=E[FinalState]-E[InitialState];                    // set energy threshold in [eV]
  for (i=0;i<=n;i++) sigma[i]=S_ion(u[i]);                    // map cross section on the grid
}                                                             // end constructor
void ionization::AddTo_BE() {                                 // begin AddTo_BE
  double u_shift;                                             // local variable (shifted energy)
  const double beta=2.0;                                      // ion treated as: 1->exc, 2->ion
  for (i=0;i<=n;i++) {                                        // begin loop over grid points
    c[i]-=sigma[i]*u[i]*N[InitialState];                      // add contribution to BE coeff.
    u_shift=beta*u[i]+threshold;                              // shifted energy in [eV]
    d[i]-=sqr(beta)*y_shift(sigma,u_shift)*u_shift*y_shift(f,u_shift)*N[InitialState];
    //d[i]-=Get_Integral(i);                                  // add contribution to BE coeff.
    QN[i]+=sigma[i]*N[InitialState];                          // add contribution to sigma(u)*N
  }                                                           // end loop over grid points
}                                                             // end AddTo_BE
void ionization::get_rates() {                                // begin get_rates
  for (i=0;i<=n;i++) tmp[i]=sigma[i]*u[i]*f[i];               // set array tmp
  k=Simpson(tmp)*v1/Ne;                                       // rate coefficient in [cm^3/s]
  R=k*N[InitialState]*Ne;                                     // rate in [cm^-33/s]
  P=R*threshold;                                              // power loss in [eV*cm^-3/s]
  loss[InitialState]+=R;                                      // species loss
  gain[FinalState]+=R;                                        // species gain
  for (i=0;i<=n;i++) tmp[i]=u[i]*sqrt_u[i]*sigma[i]*f1[i];    // set array tmp[i]
  I=-sqr(v1)*Simpson(tmp)*N[InitialState]/3.0;                // momentum loss in [cm/s^2]
}                                                             // end get_rates
ionization *ion = new ionization[MaxIon+1];                   // class variables
```

```
class electric_field {                                        // begin class definition
  private:                                                    // private variables
    int i;                                                    // counter
    double *tmp;                                              // temporary array
  public:                                                     // public variables
    double P,Be,De,Vd,sigma,J;                                // power in [eV*cm^-3/s], etc.
    double I;                                                 // momentum in [cm/s^2]
    electric_field() {tmp = new double[n+1];};                // constructor
    ~electric_field() {};                                     // destructor
    void AddTo_BE();                                          // add data to BE
    void get rates();                                         // calculate rates
};                                                            // end class definition
void electric_field::AddTo_BE() {                             // begin AddTo_BE
  for (i=0;i<=n;i++) a[i]+=u[i]*sqr(Field)/(3.0*QN[i]);       // add contribution to BE coeffi.
}                                                             // end AddTo_BE
void electric_field::get_rates() {                            // begin get_rates
  for (i=0;i<=n;i++) tmp[i]=u[i]*f[i]/QN[i];                  // set array tmp
  De=Simpson(tmp)*v1/(3.0*Ne);                                // diffusion coeff. in [cm^2/s]
  for (i=0;i<=n;i++) tmp[i]=u[i]*f1[i];                       // set array tmp
  Vd=-Simpson(tmp)*v1/(3.0*Ne);                               // drift velocity in [cm/s]
  Be=Vd/Field;                                                // mobility in [V/(cm*s)]
  P=Be*sqr(Field)*Ne;                                         // power gain in [eV*cm^-3/s]
  sigma=e0*Ne*Be;                                             // conductivity in [S/m]
  J=sigma*Field;                                              // current density in [A/cm^2]
  I=0.5*sqr(v1)*Field*Ne;                                     // momentum gain in [cm/s^2]
}                                                             // end get_rates
electric_field *el_field = new electric_field;                // class variables
```

```
class Coulomb_collisions {                                    // ----------- begin class -----------
  private:                                                    // private variables
    int i;                                                    // local counter
    double *fi1,*fi2,*fi3;                                    // arrays fi1,fi2 and fi3
    double G ee;                                              // Gee
    double ln_L;                                              // ln(L)
  public:                                                     // public variables
    double *A,*B;                                             // arrays "A" and "B"
    Coulomb_collisions();                                     // default constructor
    ~Coulomb_collisions() {};                                 // destructor
    void get_AB();                                            // calculate coefficients A(u) & B(u)
```

```
    void AddTo_BE();                                          // add coeff. to Boltzmann equation
};                                                            // ------------ end class ------------
Coulomb_collisions::Coulomb_collisions() {                    // begin constructor
  A=new double[n+1];B=new double[n+1];                        // reserve memory for arrays "A" & "B"
  fi1=new double[n+1];fi2=new double[n+1];fi3=new double[n+1]; // reserve memory for arrays fi
}                                                             // end constructor
void Coulomb_collisions::get_AB() {                           // begin "Get_AB"
  int k;double s1,s2,s3,Ne,mean_energy_density;               // local variables
  for (i=0;i<=n;i++) {                                        // begin loop over "i"
    fi1[i]=u[i]*sqrt_u[i]*f[i];                               // fi1[i]
    fi2[i]=f[i];                                              // fi2[i]
    fi3[i]=sqrt_u[i]*f[i];                                    // fi3[i]
  }                                                           // end loop over "i"
  Ne=Simpson(fi3);                                            // Ne
  mean_energy_density=Simpson(fi1);                           // <u>
  A[n]=mean_energy_density;B[n]=Ne;                           // set A[n] & B[n]
  A[n-1]=A[n];B[n-1]=B[n];                                    // set A[n-1] & B[n-1]
  A[0]=0.0;B[0]=0.0;                                          // set A[0] & B[0]
  for (k=0;k<=1;k++) {                                        // loop over "k", k=0,1
    s1=0.0;s2=0.0;s3=0.0;                                     // set to zero
    for (i=n-2-k;i>=2-k;i-=2) {                               // loop over "i"
      s1+=(fi1[i]+4.0*fi1[i+1]+fi1[i+2])*du/3.0;              // accumulate "s1"
      s2+=(fi2[i]+4.0*fi2[i+1]+fi2[i+2])*du/3.0;              // accumulate "s2"
      s3+=(fi3[i]+4.0*fi3[i+1]+fi3[i+2])*du/3.0;              // accumulate "s3"
      A[i]=mean_energy_density-s1+u[i]*sqrt_u[i]*s2;          // A[i]
      B[i]=Ne-s3;                                             // B[i]
    }                                                         // end loop over "i"
  }                                                           // end loop over "k"
}                                                             // end "Get_AB"
void Coulomb_collisions::AddTo_BE() {                         // begin "AddTo_BE"
  get_AB();                                                   // calculate arrays "A" and "B"
  ln_L=23.5+1.5*log(Te)-0.5*log(Ne);                          // calculate ln(L)
  ln L=max(ln L,1.0);                                         // limit ln L
  G_ee=4.34e-14*ln_L;                                         // calculate parameter G_ee
  for (i=0;i<=n;i++) {                                        // begin loop over "i"
    a[i]+=2.0*G_ee*A[i];                                      // add contribution to a[i]
    b[i]+=3.0*G_ee*B[i];                                      // add contribution to b[i]
  }                                                           // end loop over "i"
}                                                             // end "AddTo_BE"
Coulomb_collisions *ee_coll = new Coulomb_collisions;         // define class variable
```

```
class diffusion {
  protected:
    int i;
  public:
    double nu_dif;
    double R;
    double P;
    diffusion() {};
    void AddTo_BE();
    void get_rates();
};
void diffusion::AddTo_BE() {
  nu_dif=sqr(2.405/R0)*Damb;
  for (i=0;i<=n;i++) c[i]-=nu_dif/v1*sqrt_u[i];
}
void diffusion::get_rates() {
  mean_energy=get_mean_energy();
  R=nu_dif*Ne;
  P=R*mean_energy;
}
diffusion *dif = new diffusion;
```

```
class diss_recombination {                                  // ---------- begin class ------------
  private :                                                 // private variables
    int i;                                                  // counter
    double *tmp;                                            // temporary array
  public:                                                   // public variables
    species InitialState;                                   // initial state
    species FinalState1;                                    // final state
    species FinalState2;                                    // final state
    double *sigma;                                          // cross section in [cm^2]
    double k;                                               // rate coefficient in [cm^3/s]
    double R;                                               // rate in [cm^(-3)/s]
    double P;                                               // power loss in [eV*cm^(-3)/s]
    diss_recombination() {};                                // default constructior
    diss_recombination(species   InitialState,species   FinalState1,species   FinalState2,double
S(double u));
    ~diss_recombination() {};                               // destructior
    void AddTo_BE();                                        // add coeffi. to Boltzmann equatuion
    void get_rates();                                       // calculate rates
};                                                          // ----------- end class -------------
diss_recombination::diss_recombination(species     InitialState,species     FinalState1,species
FinalState2,double S(double u)) {
  sigma = new double[n+1];                                  // reserve memory for array "sigma"
  tmp = new double[n+1];                                    // reserve memory for array "tmp"
  this->InitialState=InitialState;                          // set InitialState
  this->FinalState1=FinalState1;                            // set FinalState
  this->FinalState2=FinalState2;                            // set FinalState
  for (i=0;i<=n;i++) sigma[i]=S(u[i]);                      // map cross section on grid
}                                                           // end constructor
void diss_recombination::AddTo_BE() {                       // begin "AddTo_BE"
  for (i=0;i<=n;i++) {                                      // loop over "i"
    c[i]-=sigma[i]*u[i]*N[InitialState];                    // add contribution to c[i]
    QN[i]+=sigma[i]*N[InitialState];                        // add contribution to sigma(u[i])*N
  }                                                         // end loop over "i"
}                                                           // end "AddTo_BE"
void diss_recombination::get_rates() {                      // begin "get_rates"
  for (i=0;i<=n;i++) tmp[i]=sigma[i]*u[i]*f[i];             // set array "tmp"
  k=Simpson(tmp)*v1;                                        // rate coefficient in [cm^3/s]
  R=k*N[InitialState]*Ne;                                   // rate in [cm^(-3)/s]
  for (i=0;i<=n;i++) tmp[i]=sigma[i]*sqr(u[i])*f[i];        // set array "tmp"
  P=Simpson(tmp)*v1*Ne*N[InitialState];                     // power loss in [eV*cm^(-3)/s]
  loss[InitialState]+=R;                                    // loss for initial state
  gain[FinalState1]+=R;                                     // gain for final state #1
  gain[FinalState2]+=R;                                     // gain for final state #2
}                                                           // end "get_rates"
diss_recombination *dr = new diss_recombination[MaxDissRec+1];    // define class variable
```

5. ACKNOWLEDGEMENTS

We would like to acknowledge valuable contributions from many collaborators and collaborators at NRL: Drs. S. Walton, D. Boris, E. Lock, M. Wolford, S. Obenschain, J. Giuliani, J. Apruzese, M. Johnson and J. Sethian.

6. PUBLICATIONS IN PEER-REVIEWED JOURNALS

1. G. M. Petrov, Tz. B. Petrova, and J. R. Peñano, "Time-dependent electron Boltzmann equation for hypersonic plasmas", Phys. Plasmas **31**, 073514 (2024)
2. Tz. B. Petrova, D. R. Boris, G. M. Petrov, M. J. Johnson, S. G. Walton, "One-dimensional, hybrid hydrodynamics-Boltzmann model for electron beam generated plasmas produced in low pressure argon", Phys. Plasmas **31**, 043515 (2024)
3. M. A. Ridenti, J. de Amorim, A. Dal Pino, V. Guerra and G. Petrov, "Causes of plasma column contraction in surface-wave-driven discharges in argon at atmospheric pressure", Phys. Rev. E **97**, 013201 (2018)
4. Tz. B. Petrova, G. M. Petrov, D. R. Boris, and S. G. Walton, "Non-equilibrium steady-state kinetics of He-air atmospheric pressure plasmas", Phys. Plasmas **24**, 013501 (2017)
5. Tz. B. Petrova, G. M. Petrov, M. F. Wolford, J. L. Giuliani, H. D. Ladouceur, F. Hegeler, M. C. Myers, and J. D. Sethian, "Effective NOx remediation from a surrogate flue gas using the US NRL Electra electron beam facility", Phys. Plasmas **24**, 023501 (2017)
6. G. M. Petrov, M. F. Wolford, Tz. B. Petrova, J. L. Giuliani, and S. P. Obenschain, "Production of radical species by electron beam deposition in an ArF* lasing medium", J. Appl. Phys. **122**, 133301 (2017)
7. G. M. Petrov, J. P. Apruzese, Tz. B. Petrova, and M. F. Wolford, "Investigation of industrial-scale carbon dioxide reduction using pulsed electron beams", J. Appl. Phys. **119**, 103303 (2016)
8. E. H. Lock, Tz. B. Petrova, G. M. Petrov, D. R. Boris, and S. G. Walton, "Electron beam-generated Ar/N_2 plasmas: the effect of nitrogen addition on the brightest argon emission lines", Phys. Plasmas **23**, 043518 (2016)
9. S. G. Walton, D. R. Boris, S. C. Hernandez, E. H. Lock, Tz. B. Petrova, G. M. Petrov, and R. F. Fernsler, "Electron Beam Generated Plasmas for Ultra Low Te Processing", ECS J. of Solid State Sc. Tech. **4**, N5033-N5040 (2015)
10. G. M. Petrov, D. R. Boris, E. H. Lock, Tz. B. Petrova, R. F. Fernsler, and S. G. Walton, "The influence of magnetic field on electron beam generated plasmas", J. Phys. D: Appl. Phys. **48**, 275202 (2015)
11. G. M. Petrov, D. R. Boris, Tz. B. Petrova, E. H. Lock, R. F. Fernsler, and S. G. Walton, "Controlling the electron energy distribution function of electron beam generated plasmas with molecular gas concentration: part II. Numerical modeling", Plasma Sources Sci. Tech. **22**, 065005 (2013)
12. Tz. B. Petrova, H. D. Ladouceur, and A. P. Baronavski, "Numerical modeling of the electrical breakdown and discharge propertiesof laser-generated plasma channels", Phys. Rev. E **76**, 066405 (2007)
13. Tz. B. Petrova, H. D. Ladouceur and A. P. Baronavski, "Nonequilibrium dynamics of laser-generated plasma channels", Phys. Plasmas **15**, 053501 (2008)
14. G. M. Petrov, J. L. Giuliani, J. P. Apruzese, A. Dasgupta, Tz. Petrova, K. Bartschat, and D. Rose, "Electron kinetics of the e-beam pumped Ar-Xe laser", J. Phys. D **40**, 4532-4543 (2007)
15. G. M. Petrov and J. Giuliani, "Inhomogeneous model of an Ar-Hg direct current column discharge", J. Appl. Phys. **94**, 62-75 (2003)
16. G. M. Petrov, J. Giuliani, and A. Dasgupta, "Electron energy deposition in an electron beam pumped KrF amplifier: Impact of beam power and energy", J. Appl. Phys. **91**, 2662-2677 (2002)
17. J. Giuliani, G. M. Petrov, and A. Dasgupta, "Electron energy deposition in an electron beam pumped KrF amplifier: Impact of the gas composition", J. Appl. Phys. **92**, 1200-1206 (2002)
18. G. M. Petrov and Ts. Petrova, A strict description of the electron energy distribution function near absorbing boundaries, IEEE Trans. Plasma Science **28**, 2202-2206 (2000)
19. Ts. Petrova and G. M. Petrov, "A strict description of the electron kinetics in rare gases near absorbing boundaries", Physica Scripta **61**, 102-111 (2000)

7. REFERENCES

1. G. J. M. Hagelaar and L. C. Pitchford, "Solving the Boltzmann equation to obtain electron transport coefficients and rate coefficients for fluid models", Plasma Sources Sci. Technol. **14**, 722–733 (2005)
2. A. Tejero-del-Caz, V. Guerra, D. Gonçalves, M. Lino da Silva, L. Marques, N. Pinhão, C. D. Pintassilgo, and L. L. Alves, "The LisbOn KInetics Boltzmann solver", Plasma Sources Sci. Technol. **28**, 043001 (2019).
3. G. L. Braglia, J. Wilhelm and R. Winkler, "Monte Carlo and Boltzmann Calculations of the Time-Dependent Velocity Distribution of Electrons in an Electric Field in a Gas", IL Nuovo Cimento **16**, 411-415 (1994)
4. Tz. B. Petrova, H. D. Ladouceur and A. P. Baronavski, "Nonequilibrium dynamics of laser-generated plasma channels", Phys. Plasmas **15**, 053501 (2008)
5. G. M. Petrov, Tz. B. Petrova, and J. R. Peñano, "Time-dependent electron Boltzmann equation for hypersonic plasmas", Phys. Plasmas **31**, 073514 (2024)
6. U. Kortshagen and H. Schluter, **"**On the influence of Coulomb collisions on the electron energy distribution function of surface wave produced argon plasmas", J. Phys. D Appl. Phys. **25**, 644-651 (1992).
7. H. Leyh, D. Loffhagen, and R. Winkler, "A new multi-term solution technique for the electron Boltzmann equation of weakly ionized steady-state plasmas", Computer Phys. Comm. **113**, 33-48 (1998); D. Loffhagen and R. Winkler, "A new nonstationary Boltzmann solver in self-consistent modelling of discharge pumped plasmas for excimer lasers", J. Comp. Phys. **112**, 91-101 (1994)
8. G. M. Petrov, D. R. Boris, Tz. B. Petrova, E. H. Lock, R.F. Fernsler, and S. G. Walton, "Controlling the electron energy distribution function of electron beam generated plasmas with molecular gas concentration: part II. Numerical modeling", Plasma Sources Sci. Tech. **22**, 065005 (2013).